

Práctica: Evolucionando el Asistente FlightAI

Introducción

Hasta ahora, habéis creado un asistente de IA capaz de consultar precios de billetes utilizando una herramienta (Tool). Sin embargo, en el mundo real, los asistentes virtuales necesitan manejar múltiples herramientas simultáneamente para resolver distintos tipos de problemas.

El Desafío: Los clientes de FlightAI están frustrados porque, aunque saben el precio, no pueden saber si su vuelo saldrá a tiempo. Vuestra misión es actualizar el código del asistente para integrar una segunda herramienta: Consulta de Estado de Vuelos.

Objetivos

1. Definir una nueva función en Python para simular una base de datos de vuelos.
2. Crear el esquema JSON para enseñar a la IA a usar esta nueva función.
3. Modificar la lógica del programa (`handle_tool_call`) para que el asistente sepa **elegir automáticamente** qué herramienta usar según la pregunta del usuario.

Pasos de la Práctica

Paso 1: Crear la "Base de Datos" de Vuelos

En vuestro código, justo debajo del diccionario `ticket_prices`, añadid un nuevo diccionario que simule el estado de los vuelos en tiempo real.

Copiad y pegad esto:

Python

```
# Base de datos simulada de estados de vuelos
```

```
flight_status_db = {  
    "FA101": "On Time",  
    "FA202": "Delayed 2 hours",  
    "FA303": "Cancelled",  
    "FA404": "Boarding"  
}
```

Paso 2: Crear la Lógica (Función Python)

Necesitamos una función que busque en esa base de datos. Cread una función llamada `get_flight_status` que reciba un `flight_number` (número de vuelo).

- **Requisito:** Si el vuelo existe, devuelve su estado. Si no existe, debe devolver el texto "Status Unknown".
- **Ayuda:** Podéis usar `.get()` igual que hicisteis con los precios.

Paso 3: Definir la Herramienta para la IA

Ahora debemos decirle a OpenAI que esta función existe. Cread un diccionario llamado `status_function`.

Poned mucha atención a la estructura:

- **name:** Debe ser exactamente `"get_flight_status"`.
- **description:** Explicad a la IA para qué sirve (ej: *"Get the current status of a specific flight"*).
- **parameters:** La propiedad necesaria es `flight_number` (tipo string).

Paso 4: Registrar la Herramienta

Buscad la lista `tools` en vuestro código. Actualmente solo tiene la función de precios. Añadid vuestra nueva función a la lista.

Debería quedar algo así:

Python

```
tools = [  
    {"type": "function", "function": price_function},  
    {"type": "function", "function": status_function} # <-- Tu nueva herramienta  
]
```

Paso 5: El Cerebro (Gestión de Múltiples Herramientas)

Este es el paso más importante. Actualmente, vuestra función `handle_tool_call` asume que el usuario *siempre* pregunta por precios. Tenéis que modificarla para usar una estructura condicional (if/elif).

La IA nos dirá qué función quiere ejecutar en `message.tool_calls[0].function.name`.

Esqueleto del código a completar:

Python

```
def handle_tool_call(message):  
    tool_call = message.tool_calls[0]  
    function_name = tool_call.function.name # ¿Qué herramienta pide la IA?  
    arguments = json.loads(tool_call.function.arguments)  
  
    response_content = {} # Aquí guardaremos el resultado  
  
    # 1. Si pide PRECIOS  
    if function_name == "get_ticket_price":
```

```

    city = arguments.get('destination_city')

    price = get_ticket_price(city)

    response_content = {"destination_city": city, "price": price}

# 2. Si pide ESTADO DE VUELO (¡Completad esta parte!)
elif function_name == "get_flight_status":

    # Extraed el número de vuelo de los argumentos...

    # Llamad a vuestra función get_flight_status...

    # Guardad el resultado en response_content...


# Empaquetamos la respuesta para OpenAI
response = {

    "role": "tool",

    "content": json.dumps(response_content),

    "tool_call_id": tool_call.id

}

return response, str(response_content)

```

Pruebas de Validación

Reiniciad vuestro programa y realizad las siguientes preguntas en el chat para verificar que funciona:

1. **Prueba de Regresión (¿Sigue funcionando lo viejo?):**
 - *Usuario:* "How much is a ticket to Tokyo?"
 - *Resultado esperado:* Debe dar el precio correcto (\$1400).
2. **Prueba de la Nueva Funcionalidad:**
 - *Usuario:* "Is flight FA202 on time?"
 - *Resultado esperado:* Debe responder: "No, flight FA202 is delayed 2 hours."
3. **Prueba Combinada (Opcional - Nivel Avanzado):**
 - *Usuario:* "Check the status of flight FA303 and tell me the price to Berlin."

- *Resultado esperado:* La IA debería ser capaz de hacer dos llamadas o gestionar la información para responder a ambas cosas (esto depende del modelo usado, gpt-4o-mini suele hacerlo bien).